CORE

MASTER SPECIFICATION — FULL DETAIL

Version 3.5 • 17 August 2026

Technical • Functional • Architectural • API • Security • Database • Administration • Governance

Document role	Primary CORE master reference for architecture, development handover and governance
Architecture status	Shared foundation; application-specific business logic remains outside CORE
Priority 17	Feature schema applied; security privilege remediation OPEN
RT-5	Resolved and committed
Dashboard	Share/Referral reposition committed and pushed; online visual/interactive verification outstanding
Payments	Stripe established; PayPal SE-5 deferred pending empirical verification
Known low issue	RT-3: four pre-existing TypeScript errors
Development rule	Reuse Before Create; no duplicate shared systems

Critical Current Security Status

RESOLVED — Priority 17 function privilege remediation was completed and verified in the current CORE deployment.
Live verification found explicit EXECUTE grants to anon/authenticated on public-schema SECURITY DEFINER functions despite migration-level REVOKE PUBLIC statements. The same default ACL behavior was found on two pre-existing functions. A corrective migration and live ACL/default-privilege verification are required before closure.

How to Use This Document

Use this document as the architectural source of truth. Use the implementation source code for exact signatures and runtime behavior. Use API_CONTRACT.md for the frozen public API contract. Use the audit documents for finding-level evidence and remediation history.

Table of Contents

CORE must remain generic for current and future applications. Shared CORE logic must not branch on hardcoded application names, slugs or application-specific business concepts. New applications must use the existing shared identity, Premium, payment, entitlement, capability, notification, messaging and API infrastructure without requiring application-specific business logic to be added to CORE.

The entitlement layer remains shared and generic. Application-specific entitlements are isolated by application context, while Global Premium remains global/user-scoped.

Application-specific products such as Listing and Sponsored are generic commercial products. A product may optionally require another entitlement. Dependency checks are generic and application-scoped; CORE does not hardcode concepts such as 'Sponsored requires Listing'.

Premium duration and price are configuration-driven. The currently supported duration values are 1, 3, 6 and 12 months; an application may offer only selected durations. €4.90 is an illustrative price, not a hardcoded CORE price.

An application may sell Global Premium or may have Premium sales disabled while still recognizing Global Premium users and providing application-specific Premium benefits. Application-specific commercial products remain independent from Global Premium.

Applications may provide additional application-specific Premium benefits to Global Premium members. The application owns the meaning and business logic of those benefits; CORE provides the shared Premium status and infrastructure.

Global Premium is user-level and global across CORE-connected applications. An active Premium membership is recognized across applications; Premium is not a separate membership system per application.

Every CORE-connected application follows the universal PRE_LAUNCH → PUBLIC launch-access model. CORE itself is explicitly excluded from this application launch gate.

A newly developed CORE-connected application defaults to PRE_LAUNCH.

In PRE_LAUNCH, the application's main production domain displays only its configurable Pre-Launch Front Page to normal public visitors.

Protected application routes remain inaccessible to normal public visitors and return them to the application's main-domain root.

The authorized application Admin retains full access during PRE_LAUNCH.

Administrators may explicitly authorize selected test users for pre-launch testing.

The application becomes public only through an explicit Admin-controlled transition from PRE_LAUNCH to PUBLIC.

The transition does not change the application's production domain, URL structure, routes, links or architecture.

The launch mechanism is generic and must apply to all current and future CORE-connected applications without application-name-specific branching.

01. Document Control & Purpose

This is the master technical, functional and architectural specification for CORE. It consolidates the current CORE professional specification, the earlier detailed CORE specification, Dashboard specification changes, Priority 16 implementation/audit material, Priority 17 implementation/correction/production verification, RT-5 verification and the latest deployment report. It is intended to be the primary reference for future CORE development, code-review decisions, application integration and architecture governance.

- Version 3.5 — 17 August 2026.
- Scope: current architecture, implemented modules, defined behavior, data domains, API surface, security model, administration, commercial model, lifecycle rules and open/deferred work.
- Terminology is preserved from the source specifications.
- Architecture-target statements are not silently converted into production claims.
- Unverified browser/production checks remain explicitly unverified.

02. Current Executive Status

CORE is the central shared SaaS platform and operating layer for current and future applications. It is the shared foundation; individual applications remain responsible for their own business-specific logic and databases.

- Priority 16: committed and verified.
- Priority 17 feature migrations: applied to production.
- RT-5: fixed and committed.
- CORE Dashboard Share/Referral reposition: implemented, committed as 26ec49a and pushed to origin/main; online visual and interactive verification remains outstanding.
- Priority 17 security privilege remediation: OPEN. Live production verification found unintended EXECUTE privileges for anon/authenticated on SECURITY DEFINER functions in the public schema. The same project-wide default-privilege behavior was also found on two pre-existing functions.
- SE-5 PayPal custom/custom_id mapping: deferred until the first real PayPal product exists and a controlled payment test is possible.
- SE-19: deferred security review.
- RT-3: four known pre-existing TypeScript errors.
- No new CORE architecture priority is currently warranted by the latest documented state.

03. Core Architectural Definition

CORE is the shared operating system for a family of applications. It centralizes functionality that must have a single source of truth across applications. It must not absorb application-specific business domains.

- Shared identity and account foundation.
- Shared application registry and capability model.
- Shared Premium/subscription/entitlement state.
- Shared Rewards/Event Engine.
- Shared Members.
- Shared Notifications/Support and CORE-wide messaging.
- Shared Advertising and commercial dashboard surfaces.
- Shared Offers/Coupons.
- Shared Administration, Security and Audit.
- Shared API, UI and localization.
- Application-specific business logic remains outside CORE.

04. Non-Negotiable Architecture Principles

The following rules govern every future CORE change.

- Single Source of Truth: shared identity, subscription, entitlement, reward and platform records resolve centrally.
- Reuse Before Create: extend existing services/entities before creating alternatives.
- Modular Architecture: each domain remains separable and reusable.
- Application Agnostic: no application-specific business rules inside shared CORE services.
- Configuration First: applications, capabilities, advertising channels, pricing and offers are configuration-driven where appropriate.
- Server-Side Authority: prices, permissions, entitlement state, rewards and payment state are validated server-side.
- Backward Compatibility: extensions preserve existing contracts unless a deliberate versioned change is approved.
- Extensibility: adding an application must not require redesigning shared core models.
- Security by enforcement: UI visibility is never an authorization boundary.
- Production discipline: diagnose → migrate → live verify → document for database changes.

05. CORE ↔ Application Boundary

CORE owns platform capabilities. Each application owns its business domain and application-specific database.

- CORE: identity, users, profiles, Applications Registry, roles/permissions, Premium/subscriptions, billing, Rewards, Advertising, Offers/Coupons, notifications, security, audit, API, shared UI and localization.
- BosniaFans: posts, comments, fan clubs, media, Shop and other BosniaFans-specific business logic.
- Svadba.ba: vendors, wedding planning, bookings, reviews and other wedding-specific logic.
- Ticketaria: events, tickets, orders, QR check-in, venues, organizers and event-specific logic.
- Gradovi.ba: cities, businesses, categories, local promotions, listings and city-specific logic.
- CORE must not contain application-specific entities such as BosniaFans Shop products or Svadba bookings.
- Shared CORE code must not branch on application name merely to implement application business behavior.

06. Application Context & Isolation

Every consuming application is registered in CORE and participates through application context, app_id and capabilities.

- Central application identification.
- Application name, slug, domain and branding metadata.
- Application enable/disable lifecycle.
- Application capabilities.
- User-to-application membership/settings.
- Application-specific subscription configuration.
- Application-aware shared links.
- Application-specific business data remains isolated.
- Shared Dashboard mechanisms must resolve the current application rather than hardcoding BosniaFans or another application.

07. Module Map

The current CORE module inventory is the authoritative high-level map.

- 01 Dashboard / Control Center.
- 02 Identity & Authentication.
- 03 Users & Profiles.
- 04 Applications Registry & Capabilities.

- 05 Roles & Permissions.
- 06 Members / Membership Directory.
- 07 Premium & Subscriptions / Entitlements.
- 08 Billing & Payments.
- 09 Universal Rewards & Event Engine.
- 10 Advertising & Campaign Distribution.
- 11 Offers & Coupons.
- 12 Notifications & Communication.
- 13 Messaging / Conversations.
- 14 Security & Audit.
- 15 API / Integration Layer.
- 16 Shared UI / Design System.
- 17 Localization.
- 18 Administration / User 360°.
- 19 Cross-cutting configuration and platform services.

08. Dashboard — Functional Specification

The CORE Dashboard is the shared authenticated control center. It must prioritize user identity, activity, rewards and user-benefit actions before commercial advertising.

- Profile/account identity.
- Primary action area: Bodovi, Aktivnost, Preporuči / Pošalji, Pozovi prijatelje / Preporuči stranicu.
- Special Offers, rendered only when offers exist.
- My Applications and existing Rewards/application content.
- Premium/subscription state where applicable.
- Activity overview.
- Payment history where applicable.
- Advertising overview/campaign management where applicable.
- Quick links.
- Footer trust/support information.
- BS/EN/DE and responsive mobile navigation.

09. Dashboard — Mandatory Share/Referral Placement

The existing ShareAndInvite and referral mechanisms are CORE functions. The required change is a Dashboard presentation change, not a new referral system.

- The four primary actions must be visible in the upper Dashboard area.
- Desktop: four visible action cards in a shared primary area; preferred arrangement is one row.
- Mobile: all four remain directly visible, preferably 2×2.
- Share/referral actions must not be menu-only, footer-only, or hidden in a secondary section.
- Existing ShareAndInvite, referral submission and application-context logic must be reused.
- No duplicate referral tables, APIs, reward rules or referral calculations.
- No BosniaFans-specific referral implementation.
- Share and invite labels must remain visible and localized.
- The Dashboard must not duplicate the same functional component in a lower section.

10. Dashboard — Offers

Offers are user benefits and should be presented prominently near the top without displacing the primary account/action area.

- Global offers can target all, Standard, Premium or other configured segments.
- Individual offers target a specific user.
- Offers have active validity windows.
- Admin controls creation, activation and targeting.
- Dashboard renders no offer when there is no applicable active offer.
- Price/discount presentation must be server-derived.
- An offer must not display a calculated price that differs from the amount actually charged.
- Current global/individual offer CTAs use supported existing checkout/pricing surfaces rather than a new universal discounted checkout engine.

11. Dashboard — Paid Advertising

Paid dashboard advertising is a separate commercial product and must remain visually secondary to the user's own Dashboard content.

- Dashboard Cards placement.
- Dashboard Featured Banner placement.
- Advertising appears lower than Profile, primary actions, Offers and core user content.
- Dashboard advertising uses the central Advertising module.
- Advertisers purchase defined impression quantities.
- Campaign delivery is fairly rotated.
- Exhausted campaigns are excluded.
- Impression recording is atomic.
- Advertising remains available through the existing Advertising navigation rather than requiring a new top-level navigation item.

12. Identity & Authentication

CORE provides one shared identity layer for all consuming applications.

- Registration/account creation.
- Login/logout.
- Google sign-in.
- Apple sign-in.
- Central session management.
- Protected routes.
- Shared user identity across applications.
- Provider-agnostic identity handling.
- Identity-lock behavior for controlled profile identity fields.
- Central authentication and authorization boundaries.
- Application-specific identity duplication is prohibited.

13. Users & Profiles

CORE maintains the shared user/profile foundation used across applications.

- Central user ID.

- Name/profile identity.
- Email.
- Avatar/profile image.
- Location.
- Language preference.
- Registration date.
- Account status.
- Membership/plan status.
- Connected applications.
- Profile visibility.
- Shared profile card.
- Application badges.
- Global Premium visibility/contact behavior where defined by CORE rules.

14. Applications Registry & Capabilities

Applications are registered once and then participate in shared CORE services through configuration.

- Application name, slug, domain and branding.
- Application status and visibility.
- Enable/disable lifecycle.
- Capabilities.
- User-to-application membership/settings.
- Application-specific subscription configuration.
- Central application identification for APIs and shared services.
- Capability gates must be evaluated server-side where they control behavior or access.
- Applications must not create parallel registries for shared CORE capabilities.

15. Roles & Permissions

Authorization is centrally controlled and enforced server-side.

- Platform roles and permissions.
- Administrative authorization.
- Server-side authorization checks.
- Application access controls.
- Admin-only mutations.
- Audit logging of privileged actions.
- RLS as the database enforcement boundary.
- Single Administrator operating model where explicitly applicable.
- Client UI state must never be treated as proof of authorization.

16. Members / Membership Directory

CORE provides shared membership configuration and presentation while allowing applications to retain their own business-specific context.

- Standard / Premium / Verified presentation.
- Central membership configuration.
- Application association without duplicating identity.
- Member-facing directory/profile integration.
- Shared membership state where defined.

- Application-specific membership context remains separate.

17. Premium, Subscriptions & Entitlements

Subscription persistence and entitlement resolution are separate concerns. User-facing Premium status is derived centrally.

- Free / Standard / Premium plans.
- Active subscription resolution.
- Start/expiry dates.
- Trials.
- Upgrade/downgrade.
- Cancellation.
- Subscription status.
- Entitlements/benefits.
- Central Premium resolution and provenance.
- Application-aware subscription records.
- Client-side plan flags must not be trusted for access decisions.

18. Billing & Payment Architecture

The current payment architecture uses configured payment links rather than a fully dynamic provider order-creation system. Stripe is the established production rail. PayPal capture/refund handling exists, while the initial custom/custom_id correlation remains deferred.

- Stripe integration.
- PayPal integration.
- Payment status and transaction/payment records.
- Payment history.
- Subscription payments.
- User/application/payment correlation.
- Billing data.
- Signed payment references.
- Webhook verification.
- Amount/plan validation.
- Refund handling.
- Payment audit trail.
- Never trust a client-submitted amount.
- Resolve product and price server-side.
- Activate subscription/entitlement only after successful verified payment event.
- Do not configure a live PayPal Payment Link until SE-5 is empirically verified.

19. Stripe Payment Rules

Stripe remains the established production payment rail in the current CORE architecture.

- Provider webhook verification.
- Server-side amount/plan verification.
- Signed payment reference correlation where required.
- Subscription activation after successful verified payment event.
- Refund handling.
- Payment state persistence.
- Auditability of payment-related state transitions.

- Client data never determines entitlement or charged amount.

20. PayPal Payment Rules

PayPal capture and refund handling exist, but the initial payment-identification mechanism using custom/custom_id is intentionally deferred because the current environment had no real configured Payment Link or credentials for empirical verification.

- Capture webhook flow exists.
- Refund handling uses provider-native payment identifier paths and does not depend on the disputed initial custom/custom_id mapping.
- SE-5 remains UNVERIFIED/DEFERRED.
- First real PayPal product must undergo a controlled payment → webhook → entitlement/subscription activation verification.
- No live PayPal Payment Link should be treated as production-ready before that verification.

21. Universal Rewards / Event Engine

CORE has one universal Rewards/Event Engine. Priority 16 hardened its rules; Priority 17 adds Reward Boosts without creating a second reward engine.

- Points balance.
- Reward ledger.
- Reward actions/events.
- Action rules.
- Daily/weekly/monthly limits.
- Proportional purchase rewards where points_per_euro is configured.
- Refund reversal ledger entries.
- Comment-created reward rules where an application emits the event.
- Premium milestones.
- Successful-invite milestone metric.
- Reward fulfillment through the existing entitlement/grant dispatcher.
- Atomic redemption where required.
- Dashboard Rewards presentation.
- Reward Boosts with action, multiplier and active time window.
- Deterministic boost selection.
- Server-side overlap validation.
- No stacking of multiple boosts where the current resolver selects one active boost.

22. Reward Boosts

Reward Boosts are configuration around the existing reward engine, not a new engine.

- Admin creates/updates a boost.
- Boost is associated with a reward action.
- Boost has multiplier and time window.
- Expired/future boosts are excluded.
- Overlapping boosts for the same action are rejected server-side using the defined half-open interval overlap test.
- Different actions may have independent windows.
- Deterministic ordering is used for active boost lookup.
- Admin authorization gates mutations.

- Database-level exclusion constraints are not currently part of the implemented design; the current rule is server-side validation under the documented Single Administrator model.

23. Advertising — Core Model

Advertising is application-agnostic and configuration-driven. CORE owns commercial campaign data and delivery logic; external automation may handle publishing/scheduling without turning CORE into an automation engine.

- Campaign management.
- Campaign lifecycle/state.
- Placements.
- Application targeting.
- External website targets.
- Social channel targets.
- Configurable channel/placement registry.
- Pricing and duration.
- Media/format rules.
- Campaign moderation.
- Campaign statistics.
- Impressions and clicks.
- Advertiser ownership/isolation.
- Dashboard advertising.
- Fair rotation.
- Impression purchase/delivery accounting.
- Atomic impression cap.
- Analytics.
- Existing payment integration.
- Future external automation integration points.

24. Advertising — Lifecycle

Commercial campaign state is separated from payment and approval decisions.

- Draft.
- Pending Payment.
- Paid.
- Pending Approval.
- Approved.
- Scheduled.
- Active.
- Paused.
- Completed.
- Rejected.
- Cancelled.
- State transitions are server-controlled and auditable.
- Payment confirmation does not automatically equal campaign approval or activation.

25. Advertising — Dashboard Impression Model

Dashboard advertising is intended to distribute purchased impressions fairly among campaigns rather than allowing one advertiser to permanently occupy the same slot.

- Advertiser purchases defined impression quantity.
- Campaign stores impressions_purchased.
- Campaign stores impressions_delivered.
- Selection excludes exhausted campaigns.
- Rotation considers delivery state.
- Impression recording is atomic at database level.
- Application-level rapid-repeat deduplication reduces repeated counting.
- Current impression metric is a delivery/fairness metric and is not itself the billing engine.
- At the current implementation, deduplication is probabilistic rather than absolute.

26. Advertising — Dashboard Placements

Two dedicated Dashboard placements exist within the central Advertising architecture.

- dashboard_cards: rotating campaign-card placement.
- dashboard_featured_banner: featured lower Dashboard banner.
- Dashboard Cards support fair rotation and impression accounting.
- Featured Banner reuses the existing active-placement creative resolver.
- Exhausted campaigns must not continue occupying Dashboard Card slots.
- Advertiser isolation is enforced through the existing advertising data-access model.
- Dashboard advertising remains visually secondary.

27. Offers — Global & Individual

Priority 17 introduced one data model with an offer_type discriminator while preserving separate conceptual admin sections for Global and Individual Offers.

- Global Offers target configured user segments such as all, Standard or Premium.
- Individual Offers target a specific user.
- Offer fields include product/reference information, discount mode, validity and activation state according to the implemented model.
- Admin controls offers centrally.
- resolveMyOffers-style server-side resolution re-derives the applicable price/discount; client values are not trusted.
- Offers render only when active and applicable.
- Global and individual offers are user-facing benefits rather than a replacement for the public coupon system.

28. Public Coupons

Public Coupons are intentionally different from personalized dashboard offers. They can be promoted publicly through Facebook, Instagram, websites and other channels.

- Public coupon is shareable by code.
- Members and non-members may discover the coupon.
- Actual redemption requires registration/authentication as defined by the current flow.
- Public /offer/:code route.
- Coupon context is preserved through authentication/onboarding.
- Supported checkout currently covers subscription_plan products.
- Unsupported product types fail clearly rather than silently pretending a discount checkout exists.
- Coupon redemption is recorded only after the supported successful payment flow.
- Usage limits are enforced server-side.
- Coupon-specific signed payment references are used in the implemented checkout architecture.

29. Coupon Atomicity & Abuse Controls

Coupon redemption must remain atomic because usage limits are shared resources.

- max_total_uses non-null: redemption attempts lock by coupon_id before the global usage check.
- max_total_uses null: existing per-(coupon,user) lock behavior remains, avoiding unnecessary cross-user serialization.
- max_uses_per_user is enforced.
- Webhook-driven redemption is post-payment.
- Idempotency behavior remains intact.
- Client-side calls must not be treated as proof of successful payment.
- Security of the underlying RPC/function grants is currently an OPEN remediation item because Priority 17 production verification found unintended anon/authenticated EXECUTE privileges.

30. Notifications & Communication

CORE provides shared notification infrastructure rather than separate notification engines in each application.

- System notifications.
- Account notifications.
- Billing notifications.
- Subscription notifications.
- Application notifications.
- Unread/read state.
- Admin-to-user communication.
- Communication administration.
- Existing notification infrastructure is reused.

31. Messaging / Conversations

CORE supports a shared 1:1 text messaging model.

- One conversation represents a canonical pair of users.
- Canonical user ordering prevents duplicate pair conversations.
- Application origin may be stored as provenance metadata.
- Text-only base scope.
- Read state.
- Cursor-oriented API pagination.
- No groups, voice/video or attachments in the defined base scope.

32. Administration / User 360°

Priority 16 established Admin User 360° as a central administration experience.

- Central admin dashboard.
- Application management.
- User search/filtering.
- User activation/deactivation.
- Verification.
- Account deletion.
- Premium management.
- Entitlement/benefit management.
- Manual points adjustment.

- Reward ledger visibility.
- Application membership visibility.
- Cross-app activity visibility.
- Audit history.
- Admin-to-user messaging.
- Communication management.
- Payment management.
- Advertising management.
- Offers/Coupons/Reward Boost administration.
- Platform configuration.

33. Security Architecture — Layers

CORE security is layered. No single layer is treated as sufficient.

- Authentication/session security.
- JWT identity.
- Authorization and protected routes.
- JWKS public-key discovery.
- Security middleware.
- CORS allowlisting based on registered application domains.
- Security headers.
- CSRF protection where applicable.
- RLS.
- Server-side validation.
- HMAC-signed payment references.
- Audit logging.
- Service-role isolation.
- No client trust for prices, entitlements, roles or payment status.

34. Security — Current Open Privilege Finding

Priority 17 production verification uncovered a project-wide Supabase default-function-privilege issue. Migration-level REVOKE PUBLIC statements did not remove separately granted anon/authenticated privileges created by the project's public-schema default ACL.

- Affected new functions: record_ad_impression() and redeem_coupon_atomic().
- Same default-privilege behavior was found on pre-existing redeem_reward_atomic() and advance_user_streak().
- record_ad_impression could be directly invoked to bypass application rate limiting and manipulate impression delivery.
- redeem_coupon_atomic could be directly invoked with arbitrary parameters and bypass the intended webhook-only payment gate.
- Required remediation: explicitly revoke EXECUTE from PUBLIC, anon and authenticated where unauthorized; preserve legitimate service_role access; correct the public-schema default privileges; live-verify ACLs/default ACL; then document closure.
- Until live verification succeeds, the finding remains OPEN and must not be described as resolved.

35. RLS & Database Security Rules

RLS is the database enforcement boundary for protected tables, but RLS does not replace correct function grants.

- Protected CORE tables must have RLS enabled where required.
- Policies must enforce ownership/role/application boundaries.

- SECURITY DEFINER functions require explicit grant review.
- Default privileges must be reviewed when creating public-schema functions.
- Service-role-only functions must not be browser-callable.
- SECURITY DEFINER functions must have controlled search_path and explicit grants.
- Client RPC exposure must be intentional and security-reviewed.
- Never assume REVOKE PUBLIC alone removes explicit role grants.

36. Audit & Compliance

CORE centralizes auditability for privileged and security-relevant operations.

- Administrative actions.
- User changes.
- Permission changes.
- Subscription changes.
- Billing events.
- Security events.
- Payment/refund events.
- Privileged reads/writes where required.
- GDPR-oriented account deletion.
- Central audit helper.
- Audit records should identify actor, action, target/context and timestamp according to the implemented audit model.

37. API Architecture

The CORE API is a thin, versioned interface over existing CORE business functions. It must not become a second business-logic layer.

- Versioned /v1 surface.
- Application authentication/context.
- Current-user operations.
- Profiles.
- Premium.
- Applications.
- Conversations/messaging.
- Notifications.
- Billing.
- Security.
- Internal administration.
- Provider-signed Stripe/PayPal webhooks.
- JWKS discovery.
- Advertising and rewards represented in the broader CORE service architecture.
- Request IDs.
- Rate limiting.
- Validation.
- Audit logging.
- Stable machine-readable errors.
- Cursor pagination.
- Allowlisted filters.
- CamelCase JSON.
- Business-object response envelopes.

38. API Surface — Current Defined Areas

The current professional specification defines the following API areas. These are architecture/documentation surfaces and must remain aligned with the actual implementation.

- /v1/auth — authentication and token operations.
- /v1/me — current-user profile/account operations.
- /v1/profiles/:username — public profile retrieval.
- /v1/premium/status — Premium status.
- /v1/premium/applications — application Premium context.
- /v1/applications — application discovery/registry.
- /v1/applications/me — current-user application associations.
- /v1/conversations — 1:1 conversation list/create.
- /v1/conversations/:id/messages — message list/create.
- /v1/conversations/:id/read — read state.
- /v1/conversations/:id/hide — conversation hide.
- /v1/notifications — notification list.
- /v1/notifications/unread-count — unread count.
- /v1/notifications/:id/read — mark read.
- /v1/notifications/read-all — mark all read.
- /v1/billing/plans — plan listing.
- /v1/billing/checkout — checkout entry.
- /v1/billing/subscriptions — subscription listing.
- /v1/billing/payments — payment listing.
- /v1/security/overview — security status.
- /v1/security/sign-out-all — global session sign-out.
- /v1/admin/* — internal administration.
- /v1/webhooks/stripe and /v1/webhooks/paypal — provider-signed webhooks.
- /.well-known/jwks.json — public key discovery.

39. Shared UI / Design System

CORE supplies reusable UI primitives and shared Dashboard structures.

- Dashboard layout.
- Sidebar.
- Mobile navigation.
- Header.
- Profile components.
- Cards.
- Buttons.
- Dialogs.
- Forms.
- Tables.
- Notifications.
- Subscription components.
- Shared visual language.
- Responsive patterns.
- Widget modularity and capability/widget gates where implemented.
- New shared UI must use the established design system rather than introduce a parallel visual system.

40. Localization

CORE is a three-language platform.

- Bosanski (BS).
- English (EN).
- Deutsch (DE).
- Central UI strings.
- Admin-visible localized configuration where appropriate.
- Consistent fallback behavior.
- New visible strings must use the i18n system.
- Dashboard Share/Referral labels must be localized.
- Existing language fallback order remains governed by the established CORE localization implementation.

41. Database Architecture

CORE uses a shared database for common platform entities while application-specific business data remains isolated in each application's database.

- Shared identity and platform entities live in CORE.
- Foreign-key relationships connect shared identity/application references.
- RLS is enabled for protected tables.
- Privileged operations use controlled server-side/service-role paths.
- Migrations are versioned.
- Historical applied migrations must never be rewritten.
- Production migration state must be checked before editing migration files.
- Generated database types should be regenerated after schema changes where practical; hand-written type approximations must not silently diverge from live schema.

42. Major CORE Data Domains

The current CORE data model is organized around shared platform domains.

- Identity/authentication.
- Profiles and Premium profile data.
- Applications/settings/capabilities.
- Roles/user roles.
- Plans/subscriptions/entitlements.
- Payments/payment references.
- Rewards ledger/rules.
- Reward milestones/boosts.
- Notifications/audit.
- Conversations/messages.
- Advertising placements/campaigns/prices.
- Dashboard offers/offer segments.
- Public coupons/coupon redemptions.
- Membership configuration/records.

43. Key Current Tables / Entities

The current documented entity inventory includes the following principal CORE tables/entities.

- applications
- profiles
- user_roles
- members_config
- user_app_settings
- subscription_plans
- subscriptions
- entitlements
- payments
- reward_ledger
- event_rules
- reward_action_rules
- reward_milestones
- reward_boosts
- notifications
- audit_logs
- ad_placements
- ad_campaigns
- ad_placement_prices
- dashboard_offers
- offer_segments
- public_coupons
- coupon_redemptions
- conversations
- messages

44. Important Server-Side Services

The current documented server/service layer includes the following central responsibilities. Exact source signatures remain implementation details and must be taken from the codebase when modifying them.

- Application resolver and capability resolution.
- Premium status resolution.
- Entitlement resolution and grants.
- Admin authorization/assertAdmin.
- User administration.
- Activity Dashboard aggregation.
- Rewards action granting and reward calculation.
- Reward boost resolution.
- Payment-reference signing and verification.
- Stripe webhook processing.
- PayPal webhook processing.
- Advertising campaign/placement services.
- Dashboard advertising rotation.
- Atomic ad impression recording.
- Atomic coupon redemption.
- Notification sending.
- Audit logging.
- Identity locking.
- Shared profile/contact rules.
- ShareAndInvite and referral submission mechanisms.

45. Payment Reference & Webhook Correlation

CORE uses signed references where the existing payment-link architecture requires correlating a provider event back to a user/application/product context.

- Payment references are HMAC-signed.
- Verification must happen server-side.
- Subscription/campaign/coupon references use the existing tagged/reference architecture.
- Provider webhook must be verified before entitlement or redemption side effects.
- Payment amount/product/plan must be validated server-side.
- Refund handling uses provider-native identifiers where implemented.
- SE-5 specifically concerns the PayPal Payment Link write-side custom parameter versus webhook custom_id correlation.

46. Offer / Coupon Lifecycle

The current offer/coupon lifecycle is explicitly server-controlled.

- Offer: Admin creates → selects audience → configures validity → active visibility → supported CTA/checkout.
- Public coupon: Admin publishes code → public /offer/:code → registration/authentication if required → supported checkout → verified webhook → atomic redemption.
- Expired/future/inactive offers do not render as active offers.
- Coupon usage limits are enforced server-side.
- Coupon redemption is post-payment in the supported flow.
- Unsupported product types fail explicitly.

47. Advertising Lifecycle & Delivery

Advertising lifecycle and delivery are separated from user dashboard presentation.

- Campaign creation.
- Payment confirmation.
- Approval.
- Scheduling.
- Activation.
- Delivery.
- Pause/completion/rejection/cancellation.
- Dashboard placement selection.
- Impression recording.
- Fair rotation.
- Exhaustion filtering.
- Statistics.
- Provider/payment reconciliation where applicable.

48. Application Lifecycle

A new application joins CORE through a controlled lifecycle rather than by copying platform modules.

- Create application.
- Register in Applications Registry.
- Configure name/slug/domain/branding.
- Configure capabilities.
- Configure application membership/settings.

- Configure plans/Premium where required.
- Configure advertising availability if required.
- Configure localization.
- Enable application.
- Application consumes shared CORE identity/services/API.
- Application retains its own business database and business logic.

49. User Lifecycle

A user has one shared CORE identity and can participate in multiple applications.

- Register/sign in once.
- CORE establishes shared identity.
- Shared profile is maintained centrally.
- User can join multiple applications.
- Application-specific membership/subscription context is associated centrally.
- Rewards, notifications and other global services operate through the shared identity.
- User can manage account, security, subscriptions and offers through CORE.

50. Commercial CORE Model

CORE contains shared monetization surfaces that can be reused across applications.

- Premium subscriptions.
- Advertising placements and impression packages.
- Dashboard advertising.
- Global and individual offers.
- Public coupons.
- Reward Boosts.
- External firms primarily purchase advertising placements rather than requiring a separate free partner-offer infrastructure.
- Dashboard Special Offers are user benefits.
- Paid advertising is visually secondary to the user's own Dashboard.

51. Production Readiness Matrix

The current state must be read as a matrix, not as a single GO/NO-GO label.

- CORE architecture: ready for application development.
- Priority 16: DONE.
- Priority 17 features/schema: APPLIED.
- RT-5: RESOLVED.
- Dashboard Share/Referral reposition: CODE DEPLOYED; online visual/interactive verification OUTSTANDING.
- Priority 17 function-privilege security remediation: OPEN.
- SE-5 PayPal mapping: DEFERRED/UNVERIFIED.
- SE-19: OPEN/DEFERRED security review.
- RT-3: LOW / four pre-existing TypeScript errors.
- Before BosniaFans production: deployment-level security-header verification, remaining required rate-limit hardening, SE-19 review, RT-3 cleanup, and controlled PayPal verification before live PayPal use.

52. Future / Deferred Scope

The following are intentional future or deferred areas and are not to be silently treated as current CORE functionality.

- Organizations.
- Feature Flags.
- External API integrations.
- Expanded Analytics.
- Marketplace integrations.
- Affiliate system.
- Advanced coupon/payment models beyond the current public-coupon implementation.
- Advanced recommendation engine.
- Moderation expansion.
- Email provider integration.
- Provider dispute/chargeback handling.
- SE-19 security review.
- SE-5 PayPal empirical verification.
- Documented Medium/Low hardening.

53. Non-Goals

CORE is not the business application layer.

- Not the BosniaFans content database.
- Not the BosniaFans Shop database.
- Not the Svadba vendor database.
- Not the Ticketaria event/ticket database.
- Not the Gradovi business-listing database.
- Not an n8n workflow engine.
- Not a social-media publishing platform.
- Not a duplicate billing system for each application.
- Not a replacement for application-specific business databases.

54. Development Workflow

Every future CORE change should follow a controlled workflow.

- 1. Identify whether the requirement is genuinely cross-application.
- 2. Search existing CORE components/services/entities before creating anything.
- 3. Identify CORE/Application boundary.
- 4. Inspect existing code and database state.
- 5. Determine whether an existing function can be extended.
- 6. Avoid duplicate architecture.
- 7. Implement the smallest correct change.
- 8. Run typecheck/build/lint and affected route checks.
- 9. If database changes exist: verify migration state before editing/applying.
- 10. Apply production migrations only with explicit authorization.
- 11. Live-verify schema, grants, RLS and affected behavior.
- 12. Update specification/audit status.
- 13. Never claim browser or production verification that was not performed.

55. Claude Code Rules — Mandatory

This section is specifically intended to govern autonomous Claude Code implementation against CORE.

- Before coding, inspect the existing implementation and documentation.
- Do not rebuild existing CORE functions when the task only changes UI placement or presentation.
- Reuse existing server functions, components, database entities and API contracts.
- Do not create application-specific logic inside CORE.
- Do not create a duplicate Referral, Rewards, Premium, Advertising, Notification or Identity system.
- Do not alter API_CONTRACT.md unless the task explicitly requires a contract change.
- Do not modify historical applied migrations.
- Do not apply database writes without explicit authorization.
- Do not silently fix unrelated findings while implementing a feature.
- Do not mark an issue resolved without the required verification.
- Do not claim browser verification without a browser.
- Do not claim payment-provider behavior is verified without an empirical provider test.
- Always separate pre-existing errors from new errors.
- Report exact files changed.
- Report database changes separately.
- Report migrations separately.
- Report commit/push status separately.

56. Acceptance Criteria for CORE Changes

A CORE change is complete only when its functional, architectural, security and verification requirements are satisfied.

- Function works in its intended path.
- Existing functionality is not duplicated or broken.
- CORE/Application boundary is preserved.
- Authorization is server-side.
- RLS/grants are correct where applicable.
- API contract remains compatible unless deliberately versioned.
- BS/EN/DE localization remains intact for visible UI.
- Responsive behavior is verified where UI changes are made.
- TypeScript/build/lint checks pass or pre-existing errors are explicitly identified.
- Production database changes are live-verified.
- Security findings introduced by the change are documented.
- Documentation reflects the actual implementation state.

57. Reference Architecture — End-to-End

The logical architecture is: User/Application UI → Application server/functions → Application business logic/database ↕ CORE API/shared services → Identity / Profiles / Applications / Premium / Rewards / Members / Offers / Advertising / Notifications / Messaging / Security / Audit. External providers and automation integrate at explicit boundaries.

- One shared identity.
- Central application registry and capabilities.
- Central Premium/subscription/entitlement resolution.
- Central Rewards/Event Engine.
- Central advertising/offers/coupons.
- Central admin/security/audit.
- Application-specific business databases remain isolated.
- External automation such as n8n may call APIs/providers but does not become CORE business logic.

58. Terminology

Core terminology used throughout the specification.

- CORE — central shared platform and operating layer.
- Application — registered product using CORE shared services while owning its business logic.
- Capability — configuration determining whether a CORE service is available to an application.
- Entitlement — resolved benefit/access granted from subscription, Premium, rewards or administration.
- Reward Event — named business event that can trigger points/benefits.
- Placement — configured location where advertising creative can be delivered.
- Campaign — commercial advertising object owned by an advertiser and distributed to targets.
- Offer — targeted dashboard promotion configured by an administrator.
- Public Coupon — shareable coupon code promoted publicly and redeemed through the supported registration/payment flow.
- SECURITY DEFINER — PostgreSQL function mode in which execution uses owner privileges; therefore grants and internal security are critical.
- Impression — recorded delivery event used for current advertising fairness/delivery accounting.

59. Governance & Freeze Rule

CORE is treated as the shared foundation for BosniaFans and future applications. The architecture is frozen except for verified cross-application requirements and explicitly authorized security/hardening work.

- A feature needed only by one application belongs in that application unless a genuine cross-application requirement is demonstrated.
- Security remediation is not considered optional feature expansion.
- New architecture requires explicit justification.
- Existing CORE functions must be reused where suitable.
- Application development may proceed against CORE subject to the explicitly listed production conditions.

60. Source Basis & Accuracy Rules

This master document is derived from the current detailed CORE professional specification and subsequent project reports. It preserves the latest known status and does not silently reconcile contradictions by guessing.

- Primary source set: CORE_Platform_Professional_Specification_v1, CORE_Final_Professional_Specification, CORE Professional Specification v2.0, CORE Dashboard specifications, Priority 16 implementation/audit reports, Priority 17 implementation/correction/production verification reports, RT-5 reports and latest Dashboard deployment verification.
- Where a source describes an architecture target rather than an implemented fact, the document treats it as a defined capability rather than claiming independent runtime proof.
- Where production/browser verification is absent, the status remains explicitly unverified.
- Where a security issue is open, it remains open until the required remediation is live-verified.

61. Mandatory Global Premium & App-Specific Commercial Product Model

This is a mandatory CORE commercial architecture rule. Global Premium and application-specific commercial products are separate entitlement classes.

- Global Premium is a platform-wide functional entitlement.
- The current reference/initial Global Premium price is 4,90 €/month. This is a planning baseline, not a permanently fixed price.

- The Global Premium price must be centrally configurable by Admin in CORE. Applications must never hardcode the Premium price.
- One active Global Premium subscription unlocks the defined Premium functions of CORE and all applications that consume the Global Premium entitlement.
- Each application may define different Premium functions. Global Premium does not require identical Premium functionality in every application.
- Application-specific Listings and Professional Products are separate from Global Premium and are not automatically included.
- Examples: Musician Listing, Studio Listing, Vendor Listing, Business Listing, Hotel/Business Listing and other directory products.
- Featured, Top Position, Sponsored, Boost, Sponsored Event and similar promotional products are separate commercial products unless explicitly configured otherwise.
- Advertising is a separate commercial product and is not automatically included in Global Premium.
- Every app-specific commercial product must have centrally configurable price, currency, billing interval, activation and lifecycle data. Prices must not be hardcoded in individual applications.
- CORE entitlements must distinguish Global Premium from application-specific product entitlements.
- A user can simultaneously have Global Premium and multiple application-specific commercial products.
- Example: a musician may have Global Premium + Musician Listing + Featured Artist + Sponsored Event as separate entitlements/purchases.
- Example: a wedding vendor may have Global Premium + Vendor Listing + Featured Vendor as separate entitlements/purchases.
- This model is mandatory for Muzika.ba, Svadba.ba, Gradovi.ba, VisitBalkan.info, Eshop.ba, Stampa.ba, Ticketaria.pro and future applications.

62. Pricing Configuration Rule

The 4,90 €/month Global Premium price is the current planning baseline only. CORE must treat it as configurable pricing data. Future changes to the Premium price must be possible without application code changes.

- CORE is the source of truth for Premium price, currency, billing interval, trial configuration and plan status.
- Application UIs retrieve the current configured price from CORE rather than embedding a numeric price in source code.
- App-specific commercial products follow the same central configuration principle.
- Price changes must follow an explicit policy for new versus existing subscriptions and must never silently rewrite historical payment records.

63. Mandatory Listing → Content/Functionality → Sponsored Visibility Model

Application-specific professional Listings are separate commercial products from Global Premium. A Listing grants the user the right to participate in the application's relevant professional directory/category and to use the Listing-specific functionality defined by that application.

- Example: a Musician Listing on Muzika.ba places the musician in the official 'Muzičari' directory/category.
- A Musician Listing may include a public professional profile and Listing-specific functions such as adding Events, where those functions are defined by Muzika.ba.
- Global Premium may provide additional functional benefits to the same musician, but does not automatically grant the Musician Listing.
- Sponsored is a separate commercial product. It is used when a listed professional/business wants increased visibility or priority placement.
- Sponsored placement may provide priority/featured positioning, a Sponsored label, rotation among sponsored positions, or other configured visibility benefits.
- Sponsored does not replace the underlying Listing. The user must have the relevant Listing/product entitlement for the sponsored placement to apply.

- The same model applies across applications: Listing establishes presence/participation; application-specific functionality defines what the Listing can do; Sponsored/Featured establishes additional paid visibility.
- CORE manages the commercial product, subscription/payment state and entitlement. The application remains responsible for its own business rules, directory structure, content model, ranking rules and exact functionality.
- Listing, Sponsored, Featured, Top Position, Boost and similar products must remain independently configurable and independently chargeable unless an explicit product configuration states otherwise.

64. Canonical Examples

- Muzika.ba: Global Premium 4,90 €/month → Premium functions; Musician Listing → presence in 'Muzičari' + Listing-defined functions such as Events; Sponsored Musician → additional paid visibility/priority.
- Svadba.ba: Global Premium 4,90 €/month → Premium functions; Vendor Listing → presence in the relevant vendor/service directory; Sponsored Vendor → additional paid visibility/priority.
- Gradovi.ba: Global Premium → Premium functions; Business Listing → presence in the business directory; Sponsored Business → additional visibility.
- VisitBalkan.info: Global Premium → Premium functions; Business/Hotel Listing → directory presence; Sponsored Business/Hotel → additional visibility.
- Ticketaria.pro: Global Premium → Premium functions; Organizer/Event commercial products → separate products; Sponsored Event → additional paid visibility.

65. Canonical Example — Muzika.ba Musician Commercial Model

66. Universal Application PRE_LAUNCH / PUBLIC Launch Standard

The following Muzika.ba scenario is a canonical reference example for the CORE commercial architecture. It is an example of how Global Premium, an application-specific Listing and Sponsored visibility are separated. It is not a requirement that every application use the same names, functions or prices.

- Global Premium — reference price 4,90 €/month: grants the platform-wide Premium entitlement and the Premium functions configured by CORE and consumed by the application. Global Premium does not automatically make the user a musician.
- Musician Listing — example price 9,90 €/month: creates a separate Musician Listing entitlement and places the user in the Muzika.ba 'Muzičari' directory. The Listing may provide a professional public profile, Events management, an inquiry/contact form, messaging and the musician designation, according to Muzika.ba business rules.
- Public visibility: a non-registered visitor sees only the basic public information permitted by Muzika.ba plus an indication that the person is a musician. Registration/authentication may be required for interaction.
- Registered Standard User: may access the interaction functions that the Musician Listing makes available to registered users, including messaging/inquiry where configured. Such interaction is not automatically a Global Premium-only function.
- Premium User: retains the same Listing access and additionally receives the Premium benefits defined by CORE and Muzika.ba.
- Sponsored Musician — example price 9,90 €/month: a separate commercial entitlement for increased visibility, such as priority placement, Sponsored labeling and configured sponsored rotation. Sponsored does not replace or include the underlying Musician Listing.
- The three commercial reasons are therefore distinct: Global Premium = functional benefits; Musician Listing = professional presence and Listing-specific functionality; Sponsored Musician = additional paid visibility.
- The example prices 4,90 €, 9,90 € and 9,90 € are configurable reference values only. Admin must be able to change them centrally without application code changes.
- This example is a model for the architecture, not a universal pricing or feature requirement. Other applications may define different Listing products, functions, prices and Sponsored products.

66. Universal Application PRE_LAUNCH / PUBLIC Launch Standard

This is a mandatory CORE standard for every application built on or connected to CORE. It does not apply to the CORE platform itself.

Every new CORE-connected application starts in PRE_LAUNCH. It remains publicly restricted until an authorized application Admin explicitly changes its launch status to PUBLIC.

PRE_LAUNCH is an access state of the same application, not a separate website or staging domain.

The application's final/main production domain is used from the beginning. The root URL '/' is the public Pre-Launch Front Page while PRE_LAUNCH.

The Pre-Launch Front Page is application-configurable and may contain the application logo, main image/banner, title, information text, development/preparation message, social profiles and optional contact information.

Normal public visitors may access only the Pre-Launch Front Page. Protected application routes such as members, events, business, shop, dashboard and other application functionality must not be publicly accessible during PRE_LAUNCH.

Direct URL access must be protected at the authoritative server-side/access-control boundary; hiding navigation or UI elements is insufficient.

The authorized application Admin retains full access to the complete application during PRE_LAUNCH for configuration, development, testing and quality control.

The Admin may explicitly authorize selected application-scoped test users to access the application before PUBLIC launch.

The application must become PUBLIC only through an explicit Admin-controlled transition. Development completion, deployment, successful builds or tests must not automatically publish the application.

Changing PRE_LAUNCH to PUBLIC must not require a domain change, URL migration, route migration, link replacement or architectural rebuild.

The launch mechanism must be generic and must never branch on a hardcoded application name, slug or business concept.

The standard must be inherited by every current and future CORE-connected application. CORE provides the shared mechanism; each application provides its own Pre-Launch content and business-specific functionality.

CORE infrastructure endpoints that legitimately need public access must not be indiscriminately blocked by the application launch gate.